operations

Operations Guide

This guide covers deployment options, monitoring, troubleshooting, and scaling for Helix Cluster OS in production.

Deployment Options

1. Binary Deployment

Build and run services directly on hosts:

```
make build
./bin/helixd --config helixd.yaml
./bin/helix-gateway --config gateway.yaml
# ... start remaining services
```

Best for: Development, bare-metal environments, or when you manage your own orchestration.

---

2. Docker Deployment

Build images and run with Docker Compose:

```
make docker-build
 docker compose -f deployments/docker-compose.yml up -d
```

Best for: Single-node demos, CI pipelines, or small staging environments.

---

3. Kubernetes Deployment

Deploy to Kubernetes using the provided manifests:

```
kubectl apply -k deployments/k8s/overlays/production/
```

Key resources:

Resource	Purpose
`etcd` StatefulSet	3-node etcd cluster with persistent volumes
`helixd` Deployment	Control plane with 2+ replicas
`helix-gateway` Deployment	Edge gateway with HPA
Service Deployments	14 service Deployments, each with HPA
Ingress	TLS-terminated external access

Best for: Production, multi-node clusters, automatic failover, and scaling.

---

4. Config-Driven Worker Deployment (persistent membership)

For bringing up a LAN/SSH fleet of persistent worker nodes, use the config-driven deploy path. Everything is derived from a single source of truth, `deploy/cluster.env` — no IPs, ports, hosts or keys are hardcoded in the scripts.

```
# 1. Declare the fleet in deploy/cluster.env (HELIX_NODE_<id>
        inventory + HELIX_NODES).
# 2. Build the linux agent into dist/, then deploy every node in
        the inventory:
./deploy/deploy-workers.sh                  # all HELIX_NODES
./deploy/deploy-workers.sh thinker nezha    # a subset
HELIX_HOST_IP=10.0.0.5 ./deploy/deploy-workers.sh   # pin the
        control-host IP
```

`deploy-workers.sh` ships the agent binary + `agent-launch.sh` launcher to each worker over ssh/scp and starts a **persistent** `helix-agent` that registers under `/clusteros/nodes/<id>` and keeps its etcd lease alive for its whole lifetime.

> **Requirement — user linger.** The agent runs as a systemd *user* service, so the deploy enables linger (`loginctl enable-linger "$USER"`) on each worker so the user manager — and the agent — survives logout/reboot. A user may enable its own linger without root. `deploy-workers.sh` does this automatically; if you launch the agent by hand, enable linger first or the agent will not persist across sessions.

The control host runs the infra stack (etcd, helixd, …) via `deploy/compose/helix_infra.yml`, which reads `deploy/cluster.env` through `--env-file`. The published etcd/kafka host ports below come from that compose file.

Best for: Persistent multi-node bare-metal/LAN fleets without Kubernetes.

Monitoring and Alerting

Helix Cluster OS exposes Prometheus metrics on `:9090/metrics` for every service.

Key Metrics

Metric	Description	Alert Threshold
`helix_request_duration_seconds`	API latency	p99 > 500ms
`helix_request_errors_total`	Error rate	> 1% over 5m
`helix_etcd_watch_latency_seconds`	etcd watch lag	> 1s
`helix_node_health`	Node health status	== 0 (unhealthy)
`helix_build_queue_depth`	Pending build jobs	> 100

Recommended Stack

- **Prometheus** for metrics collection
- **Grafana** for dashboards (dashboard JSON in `deployments/monitoring/`)
- **Alertmanager** for routing alerts
- **Loki** for log aggregation

Health Checks

Each service exposes:

- `GET /healthz` — liveness probe
- `GET /readyz` — readiness probe

Configure these in your orchestrator for automatic restart and traffic routing.

Troubleshooting Common Issues

etcd Connection Failures

Symptoms: Services fail to start, logs show `etcd: connection refused`.

Resolution: - Verify etcd endpoints: `etcdctl endpoint health` - Check TLS certificates are valid and not expired. - Ensure network policies allow traffic to etcd ports. The config-driven infra stack (`deploy/compose/helix_infra.yml`) publishes each of the 3 etcd members on its own host client port — `2379` (etcd-1), `2479` (etcd-2), `2579` (etcd-3) — each mapped to the in-container client port `2379`; peer ports are `2380`/`2381`/`2382`. Kafka exposes a dual-listener broker per member on host ports `9092` (kafka-1), `9093` (kafka-2), `9094` (kafka-3). Workers must be able to reach every published member directly.

Gateway 502 / 503 Errors

Symptoms: Clients receive gateway errors.

Resolution: - Check gateway logs for upstream connection failures. - Verify target services are registered in etcd (`helixctl service list`). - Check readiness probes on backend services.

High Build Queue Depth

Symptoms: Jobs remain pending, `helix_build_queue_depth` is high.

Resolution: - Scale build workers: `kubectl scale deployment helix-build --replicas=10` - Check scheduler logs for resource constraints. - Verify node capacity via `helixctl node list`.

Certificate Expiry

Symptoms: mTLS handshake failures between services.

Resolution: - Check certificate expiry: `openssl x509 -in cert.pem -noout -dates` - Rotate certificates (see TLS Setup).

Scaling Guidelines

Horizontal Scaling

Component	Scaling Trigger	Action
Gateway	CPU > 70% or RPS > 10k	Add gateway replicas
Build Service	Queue depth > 50	Add build workers
Scheduler	Scheduling latency > 200ms	Add scheduler replicas
etcd	Disk latency > 10ms	Add etcd nodes (odd count)

Vertical Scaling

- **Gateway:** 2 vCPU, 4 GiB memory baseline
- **Services:** 1 vCPU, 2 GiB memory baseline
- **etcd:** 4 vCPU, 8 GiB memory, SSD storage

etcd Sizing

Cluster Size	Max Nodes	Max Concurrent Jobs
1	10	100
3	500	10,000
5	5,000	100,000

> **Note:** etcd performance is sensitive to disk I/O. Use dedicated SSDs for production clusters.